

JavaScript Intermediate — `this` Keyword & Binding

Section 20 — `this` Keyword & Binding

JavaScript-এর সবচেয়ে confusing কিন্তু সবচেয়ে গুরুত্বপূর্ণ conceptগুলোর একটি হলো `this` ।

অনেক beginner ভাবে:

"`this` মানে current object."

এটা সবসময় সত্যি না ।

`this` কোন value হবে সেটা নির্ভর করে **function** কীভাবে **call** করা হয়েছে তার ওপর ।

এই section-এ আমরা শিখব:

1. `this` কী
2. `this` কীভাবে determine হয়
3. 5টি প্রধান binding rule
4. Default binding
5. Implicit binding
6. Explicit binding
7. `call()`
8. `apply()`
9. `bind()`
10. `new` binding
11. Arrow function-এর lexical `this`
12. Callbacks-এ `this`
13. Event handlers-এ `this`
14. `this` হারিয়ে যাওয়ার সমস্যা
15. Common mistakes
16. Output prediction
17. Practical patterns
18. Interview questions

1. `this` কী?

`this` হলো JavaScript-এর একটি special keyword ।

এটা সাধারণ variable-এর মতো declare করা হয় না ।

```
console.log(this);
```

`this` -এর value context এবং function call-এর ওপর নির্ভর করে।

সবচেয়ে important rule:

Regular function-এর ক্ষেত্রে `this` সাধারণত **function** কীভাবে **call** হয়েছে তার ওপর নির্ভর করে।

2. `this` Lexical Scope-এর মতো না

Lexical variable যেমন:

```
const name = "Duke";

function sayName() {
  console.log(name);
}
```

`name` কোথায় লেখা হয়েছে সেটা দেখে JavaScript determine করে।

কিন্তু regular function-এর `this` :

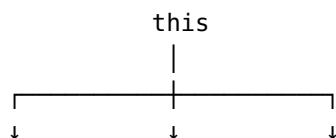
```
function show() {
  console.log(this);
}
```

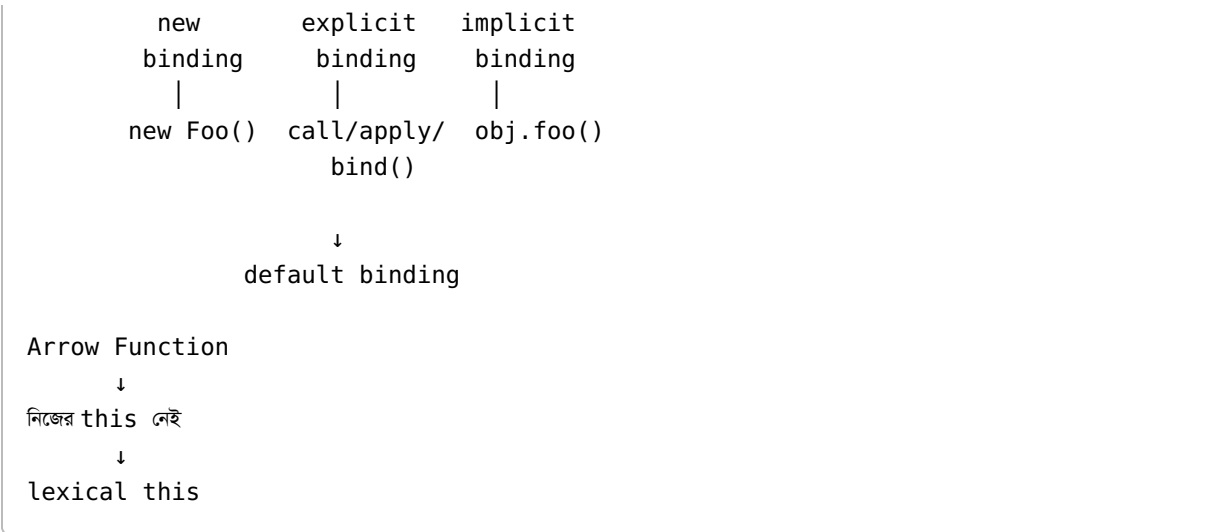
কোথায় function লেখা হয়েছে শুধু সেটা দেখে determine হয় না।

Function কীভাবে invoke করা হয়েছে সেটা গুরুত্বপূর্ণ।

3. The Five Main `this` Binding Rules

Regular function-এর `this` বোঝার জন্য এই order মনে রাখো:





আর practicalভাবে function call দেখলে এই precedence খুব useful:

1. new
2. call/apply/bind
3. object.method()
4. default

Arrow function এই rules follow করে না; সেটা surrounding lexical `this` ব্যবহার করে।

4. Rule 1 — Default Binding

যখন কোনো regular function-এর সাথে কোনো object context দেওয়া হয়নি:

```
function showThis() {  
  console.log(this);  
}  
  
showThis();
```

এটাকে standalone function call বলা যায়।

Non-strict mode

Browser-এর classic non-strict script-এ `this` সাধারণত global object-এর দিকে যেতে পারে।

Browser-এ:

```
window
```

Node.js/module context-এ behavior আলাদা হতে পারে।

Strict mode

```
"use strict";

function showThis() {
  console.log(this);
}

showThis();
```

Output:

```
undefined
```

5. Strict Mode কেন Important?

Strict mode accidental global `this` behavior prevent করে।

```
"use strict";

function test() {
  console.log(this);
}

test();
```

এখানে:

```
this → undefined
```

তাই modern JavaScript শেখার সময় strict-mode behavior বুঝে রাখা গুরুত্বপূর্ণ।

6. Rule 2 — Implicit Binding

যখন function কোনো object-এর method হিসেবে call হয়:

```
const user = {  
  name: "Duke",  
  
  sayName() {  
    console.log(this.name);  
  }  
};  
  
user.sayName();
```

এখানে:

```
this → user
```

তাই output:

```
Duke
```

7. Implicit Binding-এর Golden Rule

Call site দেখো:

```
object.method();
```

সাধারণত:

```
this → object
```

Example:

```
const user = {  
  name: "Duke",
```

```
    greet() {  
      console.log(this.name);  
    }  
  };  
  
user.greet();
```

এখানে:

```
this === user
```

8. `this` Function-এর ভিতরে **Automatically Object** না

এই code দেখো:

```
const user = {  
  name: "Duke",  
  
  greet() {  
    console.log(this);  
  }  
};
```

এখানে `this`-এর value function declaration দেখে determine হয়নি।

এটা determine হয়েছে:

```
user.greet();
```

এই call-এর কারণে।

9. Implicit Binding — One Level Deeper

```
const user = {  
  name: "Duke",  
  
  address: {  
    city: "Dhaka",
```

```
    showCity() {  
      console.log(this.city);  
    }  
  }  
};  
  
user.address.showCity();
```

এখানে:

this → user.address

তাই:

Dhaka

কারণ immediate object হলো:

user.address

10. Nested Object Confusion

অনেকে ভাবে:

this → user

কিন্তু:

user.address.showCity();

এর call-site:

```
user.address.showCity  
      ↑  
    immediate receiver
```

তাই:

```
this → user.address
```

11. Method আলাদা করে রাখলে `this` Change হতে পারে

```
const user = {  
  name: "Duke",  
  
  greet() {  
    console.log(this.name);  
  }  
};  
  
user.greet();
```

এখানে:

```
Duke
```

কিন্তু:

```
const greet = user.greet;  
  
greet();
```

এখন আর:

```
user.greet()
```

হিসেবে call হচ্ছে না।

এটা standalone call:

```
greet()
```

তাই `this` implicitভাবে `user` থাকবে না।

Strict mode-এ:

```
this → undefined
```

12. Very Important — Method Extraction

এই pattern খুব common:

```
const user = {  
  name: "Duke",  
  
  greet() {  
    console.log(this.name);  
  }  
};  
  
const fn = user.greet;  
  
fn();
```

এখানে `fn` শুধু function reference ।

Original object-এর সাথে binding automatically preserved হয় না ।

এটাই `this` -এর একটি common source of bugs ।

13. Rule 3 — Explicit Binding

আমরা manually function-কে বলতে পারি:

"এই function call-এ `this` হিসেবে এই object ব্যবহার করো ।"

এর জন্য ব্যবহার করি:

```
call()  
apply()  
bind()
```

এগুলোকে বলা হয় **explicit binding tools** ।

14. `call()`

Syntax:

```
function.call(thisArg, arg1, arg2, ...);
```

Example:

```
function greet() {  
  console.log(this.name);  
}  
  
const user = {  
  name: "Duke"  
};  
  
greet.call(user);
```

এখানে:

```
this → user
```

Output:

```
Duke
```

15. `call()` with Arguments

```
function introduce(age, profession) {  
  console.log(  
    this.name,  
    age,  
    profession  
  );  
}  
  
const user = {
```

```
    name: "Duke"
  };

  introduce.call(user, 31, "Frontend Developer");
```

Output conceptually:

```
Duke 31 Frontend Developer
```

16. apply()

`apply()` প্রায় `call()`-এর মতো।

Difference হলো arguments array-এর মতো structure-এ দেওয়া হয়।

```
function introduce(age, profession) {
  console.log(
    this.name,
    age,
    profession
  );
}

const user = {
  name: "Duke"
};

introduce.apply(
  user,
  [31, "Frontend Developer"]
);
```

17. call() vs apply()

`call()`

```
fn.call(obj, arg1, arg2, arg3);
```

apply()

```
fn.apply(obj, [arg1, arg2, arg3]);
```

মূল difference:

```
call
↓
arguments individually

apply
↓
arguments as array-like collection
```

18. bind()

`bind()` function immediately execute করে না।

বরং একটি **new function** return করে যার `this` binding set করা থাকে।

```
function greet() {
  console.log(this.name);
}

const user = {
  name: "Duke"
};

const boundGreet = greet.bind(user);

boundGreet();
```

Output:

```
Duke
```

19. call() vs bind()

call()

```
greet.call(user);
```

Function **immediately execute** করবে।

bind()

```
const newFn = greet.bind(user);
```

Function execute করবে না।

নতুন function তৈরি করবে।

তারপর:

```
newFn();
```

execute হবে।

20. apply() vs bind()

```
greet.apply(user, [31]);
```

→ এখনই execute।

```
const fn = greet.bind(user, 31);
```

→ পরে execute করার জন্য নতুন function।

21. Complete Comparison

Method	this set করে	Immediately execute?	Arguments
call()	✓	✓	আলাদা arguments
apply()	✓	✓	array-like
bind()	✓	✗	আলাদা arguments / partial args

22. Function Borrowing

`call()` দিয়ে অন্য object-এর method temporarily ব্যবহার করা যায়।

```
const user1 = {
  name: "Duke",

  greet() {
    console.log(`Hello ${this.name}`);
  }
};

const user2 = {
  name: "Rahim"
};

user1.greet.call(user2);
```

Output:

Hello Rahim

এখানে `user1`-এর method ব্যবহার হচ্ছে, কিন্তু:

this → user2

এটাকে **function borrowing** বলা হয়।

23. `bind()` দিয়ে Reusable Function

```
function greet() {  
  console.log(`Hello ${this.name}`);  
}  
  
const user = {  
  name: "Duke"  
};  
  
const greetUser = greet.bind(user);  
  
greetUser();  
greetUser();  
greetUser();
```

একই binding বারবার ব্যবহার করা যায়।

24. Partial Application with `bind()`

`bind()` শুধু `this` bind করে না, arguments-ও pre-fill করতে পারে।

```
function multiply(a, b) {  
  return a * b;  
}  
  
const double = multiply.bind(null, 2);  
  
console.log(double(5));
```

Output:

10

এখানে:

a = 2
b = 5

25. Rule 4 — new Binding

Constructor function বা class-এর সাথে `new` ব্যবহার করলে নতুন object তৈরি হয় এবং `this` সেই নতুন object-কে reference করে।

```
function User(name) {  
  this.name = name;  
}  
  
const user = new User("Duke");  
  
console.log(user.name);
```

Output:

Duke

এখানে:

this → newly created object

26. new কীভাবে কাজ করে?

Conceptually:

```
new User("Duke")  
  ↓  
নতুন object তৈরি  
  ↓  
prototype connect  
  ↓  
this → নতুন object  
  ↓  
constructor execute  
  ↓  
object return
```

তাই:

```
function User(name) {  
  this.name = name;  
}
```

এর ভিতরে `this` হলো newly created instance ।

27. Class-এর ক্ষেত্রে `this`

```
class User {  
  constructor(name) {  
    this.name = name;  
  }  
  
  greet() {  
    console.log(this.name);  
  }  
}  
  
const user = new User("Duke");  
  
user.greet();
```

এখানে:

```
constructor-এর this → new instance  
method call-এর this → user
```

28. `new` vs `call()`

দুটোতেই `this` object-এর সাথে যুক্ত হয়, কিন্তু উদ্দেশ্য আলাদা ।

`call()`

```
fn.call(obj);
```

→ existing object-কে `this` হিসেবে ব্যবহার করে ।

new

```
new Fn();
```

→ নতুন object তৈরি করে এবং সেটাকে `this` করে।

29. Rule 5 — Arrow Function

এখানেই `this` সবচেয়ে interesting।

Arrow function-এর নিজস্ব `this` নেই।

```
const greet = () => {  
  console.log(this);  
};
```

Arrow function নিজের `this` তৈরি করে না।

এটা surrounding lexical context থেকে `this` নেয়।

এটাকে বলা হয়:

Lexical `this`

30. Lexical `this` কী?

Lexical মানে:

যেখান থেকে function তৈরি হয়েছে, সেই surrounding context থেকে value পাওয়া।

Arrow function-এর ক্ষেত্রে:

Arrow Function

↓

নিজের `this` নেই

↓

outer scope-এর `this` ব্যবহার করে

31. Regular Function vs Arrow Function

Regular function:

```
const user = {  
  name: "Duke",  
  
  greet: function () {  
    console.log(this.name);  
  }  
};  
  
user.greet();
```

এখানে:

```
this → user
```

কিন্তু arrow:

```
const user = {  
  name: "Duke",  
  
  greet: () => {  
    console.log(this.name);  
  }  
};
```

এখানে arrow function নিজের `this` নেয় না।

তাই `user` automatically `this` হবে না।

32. Arrow Function Object Method হিসেবে কেন Problem হতে পারে?

এই code:

```
const user = {  
  name: "Duke",
```

```
    greet: () => {
      console.log(this.name);
    }
  };

user.greet();
```

অনেকে expected output ভাবে:

Duke

কিন্তু সেটা হবে না।

কারণ arrow function-এর:

নিজস্ব this নেই

এটা outer lexical `this` ব্যবহার করে।

33. Arrow Function-এর Main Advantage

Callback-এর মধ্যে `this` preserve করার জন্য arrow function খুব useful।

Example:

```
const user = {
  name: "Duke",

  greet() {
    setTimeout(() => {
      console.log(this.name);
    }, 1000);
  }
};

user.greet();
```

এখানে:

```
greet()
↓
this → user
↓
arrow callback
↓
নিজের this নেই
↓
outer this ব্যবহার করে
↓
user
```

Output:

Duke

34. Regular Callback-এর সমস্যা

```
const user = {
  name: "Duke",

  greet() {
    setTimeout(function () {
      console.log(this.name);
    }, 1000);
  }
};

user.greet();
```

এখানে callback হলো regular function ।

এটার নিজের `this` আছে এবং সেটা `greet()` -এর `this` automatically inherit করে না ।

তাই `this.name` expected `Duke` নাও হতে পারে ।

35. Old Solution — `self` / `that`

Arrow function আসার আগে developers প্রায়ই করত:

```
const user = {
  name: "Duke",

  greet() {
    const self = this;

    setTimeout(function () {
      console.log(self.name);
    }, 1000);
  }
};
```

এখানে:

```
self → user
```

কিন্তু modern JavaScript-এ সাধারণত arrow function cleaner solution ।

36. Another Old Solution — `bind()`

```
const user = {
  name: "Duke",

  greet() {
    setTimeout(
      function () {
        console.log(this.name);
      }.bind(this),
      1000
    );
  }
};
```

এখানে callback-এর `this` manually bind করা হয়েছে ।

37. Arrow vs `bind()`

দুটোই callback-এর context problem solve করতে পারে ।

Arrow

```
setTimeout(() => {  
  console.log(this.name);  
}, 1000);
```

Bind

```
setTimeout(  
  function () {  
    console.log(this.name);  
  }.bind(this),  
  1000  
);
```

Modern code-এ arrow সাধারণত সহজ।

38. this in Callbacks

Callbacks-এ `this` বুঝতে প্রথমে দেখতে হবে callback কোন ধরনের function।

Regular function

```
items.forEach(function (item) {  
  console.log(this);  
});
```

`this` automatically outer function-এর `this` হয়ে যাবে—এমন guarantee নেই।

Arrow function

```
items.forEach((item) => {  
  console.log(this);  
});
```

Arrow lexical `this` ব্যবহার করবে।

39. Array Method-এর `thisArg`

কিছু array methods optional `thisArg` accept করে।

যেমন:

```
const user = {
  prefix: "User"
};

const names = ["Duke", "Rahim"];

names.forEach(function (name) {
  console.log(this.prefix, name);
}, user);
```

এখানে callback-এর `this` হবে `user`।

Output:

```
User Duke
User Rahim
```

40. Arrow Function-এর সাথে `thisArg`

```
const user = {
  prefix: "User"
};

const names = ["Duke", "Rahim"];

names.forEach((name) => {
  console.log(this.prefix, name);
}, user);
```

এখানে `thisArg: user` arrow function-এর `this` change করবে না।

কারণ:

Arrow function নিজের `this` binding ignore করে lexical `this` ব্যবহার করে।

41. DOM Event Handler-এ `this`

Browser DOM-এর traditional event handler-এ regular function ব্যবহার করলে `this` সাধারণত event handler যে element-এর সাথে registered হয়েছে সেটাকে reference করে।

Example:

```
button.addEventListener("click", function (event) {  
  console.log(this);  
});
```

এখানে `this` সাধারণত:

```
button element
```

42. `this` vs `event.currentTarget`

Event handler-এ:

```
button.addEventListener("click", function (event) {  
  console.log(this);  
  console.log(event.currentTarget);  
});
```

Traditional regular callback-এর ক্ষেত্রে দুটো একই element-কে represent করতে পারে।

Conceptually:

```
this  
↓  
handler-এর current context  
  
event.currentTarget  
↓  
যে element-এর উপর listener registered
```

43. Arrow Function in Event Handler

```
button.addEventListener("click", (event) => {  
  console.log(this);  
});
```

এখানে `this` button হবে না।

কারণ arrow function নিজের `this` নেয় না।

তাই DOM event-এ element context দরকার হলে:

```
event.currentTarget
```

ব্যবহার করা বেশি explicit।

Example:

```
button.addEventListener("click", (event) => {  
  console.log(event.currentTarget);  
});
```

44. `this` vs `event.target`

এটা খুব important।

ধরো:

```
<button>  
  <span>Click Me</span>  
</button>
```

Click করা হলো `span`-এ।

Event handler:

```
button.addEventListener("click", function (event) {  
  console.log(this);  
});
```

```
console.log(event.target);  
console.log(event.currentTarget);  
});
```

তখন:

```
this  
↓  
button  
  
event.currentTarget  
↓  
button  
  
event.target  
↓  
span
```

মনে রাখো:

`target` = যেখানে event শুরু হয়েছে।

`currentTarget` = যে element-এর handler এখন execute হচ্ছে।

45. Callback + Object Method Trap

```
const user = {  
  name: "Duke",  
  
  greet() {  
    console.log(this.name);  
  }  
};  
  
["a", "b"].forEach(user.greet);
```

এখানে `user.greet` callback হিসেবে pass হয়েছে।

কিন্তু `forEach` এটাকে:

```
user.greet()
```

হিসেবে call করছে না।

তাই `this` হারিয়ে যেতে পারে।

46. Solution 1 — Arrow Wrapper

```
["a", "b"].forEach(() => {  
  user.greet();  
});
```

এখানে `user.greet()` আবার object method call হিসেবে execute হচ্ছে।

47. Solution 2 — `bind()`

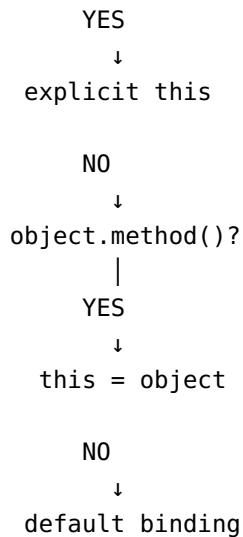
```
["a", "b"].forEach(  
  user.greet.bind(user)  
);
```

এখানে `greet`-এর `this` permanently bound function-এর মাধ্যমে set করা হয়েছে।

48. `this` Binding Priority

Regular function-এর ক্ষেত্রে এই hierarchy খুব useful:

```
Is it called with new?  
|  
YES  
↓  
this = new object  
  
NO  
↓  
call/apply/bind দিয়ে bound?  
|
```



Arrow function হলে:

Arrow function
↓
নিজস্ব this নেই
↓
lexical this

49. Important Exception — Bound Function

`bind()` করা function-এর ক্ষেত্রে bound `this` সাধারণ call-এর মাধ্যমে সহজে পরিবর্তন করা যায় না।

Example:

```
function greet() {  
  console.log(this.name);  
}  
  
const user1 = {  
  name: "Duke"  
};  
  
const user2 = {  
  name: "Rahim"  
};
```

```
const bound = greet.bind(user1);  
  
bound.call(user2);
```

Output:

Duke

কারণ `bound` already `user1` -এর সাথে bound।

50. Arrow Function + `call()`

এখানে beginner-রা প্রায়ই ভুল করে।

```
const greet = () => {  
  console.log(this.name);  
};  
  
const user = {  
  name: "Duke"  
};  
  
greet.call(user);
```

`call()` arrow function-এর lexical `this` replace করতে পারে না।

কারণ:

Arrow function-এর own `this` binding নেই।

51. Arrow Function + `bind()`

একইভাবে:

```
const greet = () => {  
  console.log(this.name);  
};  
  
const user = {
```

```
    name: "Duke"
  };

  const bound = greet.bind(user);

  bound();
```

Arrow function-এর lexical `this`-কে `bind()` দিয়ে traditional regular-function style-এ change করা যায় না।

52. Regular Function + `bind()`

Regular function:

```
function greet() {
  console.log(this.name);
}

const user = {
  name: "Duke"
};

const bound = greet.bind(user);

bound();
```

এখানে কাজ করবে।

```
this → user
```

53. `this` Inside Constructor

```
function User(name) {
  this.name = name;
}

const user1 = new User("Duke");
const user2 = new User("Rahim");
```

এখানে:

```
user1 → this for first constructor call  
user2 → this for second constructor call
```

তাই:

```
console.log(user1.name);  
console.log(user2.name);
```

Output:

```
Duke  
Rahim
```

54. `this` Inside Class Method

```
class Counter {  
  constructor() {  
    this.count = 0;  
  }  
  
  increment() {  
    this.count++;  
  }  
}  
  
const counter = new Counter();  
  
counter.increment();  
  
console.log(counter.count);
```

Output:

```
1
```

এখানে:

```
counter.increment()  
  ↓  
this → counter
```

55. Class Method Extract করলে Problem

```
class Counter {  
  count = 0;  
  
  increment() {  
    this.count++;  
  }  
}  
  
const counter = new Counter();  
  
const fn = counter.increment;  
  
fn();
```

এখানে method আর:

```
counter.increment()
```

হিসেবে call হচ্ছে না।

তাই `this` lost হতে পারে।

56. Class Method-এর জন্য `bind()`

```
class Counter {  
  count = 0;  
  
  increment() {  
    this.count++;  
  }  
}
```

```
const counter = new Counter();

counter.increment =
  counter.increment.bind(counter);
```

এখন:

```
const fn = counter.increment;

fn();
```

কাজ করবে।

57. Modern Alternative — Arrow Class Field

```
class Counter {
  count = 0;

  increment = () => {
    this.count++;
  };
}
```

এখানে arrow function lexical `this` ব্যবহার করে।

এটা callback হিসেবে method pass করার ক্ষেত্রে useful হতে পারে।

58. Common Mistake — `this` Means Parent Object

ভুল ধারণা:

`this` সবসময় parent object।

আসলে `this` call-site/context-এর rules follow করে।

Example:

```
const user = {
  name: "Duke",

  greet() {
    console.log(this.name);
  }
};

const fn = user.greet;

fn();
```

`fn()` call-এ `this` আর `user` না।

59. Common Mistake — Arrow Method

এটা সাধারণত method-এর জন্য ভুল pattern:

```
const user = {
  name: "Duke",

  greet: () => {
    console.log(this.name);
  }
};
```

যদি method-এর `this` হিসেবে object দরকার হয়, regular method syntax ব্যবহার করো:

```
const user = {
  name: "Duke",

  greet() {
    console.log(this.name);
  }
};
```

60. Common Mistake — Callback-এ Regular Function

```
const user = {
  name: "Duke",

  greet() {
    setTimeout(function () {
      console.log(this.name);
    }, 1000);
  }
};
```

Callback-এর `this` automatically outer `this` inherit করবে না।

Arrow:

```
const user = {
  name: "Duke",

  greet() {
    setTimeout(() => {
      console.log(this.name);
    }, 1000);
  }
};
```

এখানে lexical `this` কাজ করে।

61. Common Mistake — `const self = this`

এটা technically wrong না:

```
const self = this;
```

কিন্তু modern JavaScript-এ unnecessary হলে arrow function prefer করা যায়।

```
setTimeout(() => {  
  console.log(this.name);  
});
```

আর React functional components-এ সাধারণত এই ধরনের `this` pattern প্রয়োজনই হয় না।

62. `this` in Modern React

Modern React functional components-এ সাধারণত class component-এর মতো `this` ব্যবহার করা হয় না।

Example:

```
function User() {  
  const name = "Duke";  
  
  return <h1>{name}</h1>;  
}
```

এখানে:

```
this.name
```

এর প্রয়োজন নেই।

State/props/local variables সরাসরি ব্যবহার করা হয়।

Class component-এ:

```
class User extends React.Component {  
  render() {  
    return <h1>{this.props.name}</h1>;  
  }  
}
```

এখানে `this` গুরুত্বপূর্ণ।

63. `this` Prediction Strategy

কোনো code-এ `this` দেখলে panic করবে না।

এই 5-step process follow করো:

Step 1

Function regular নাকি arrow?

```
arrow → lexical this  
regular → next steps
```

Step 2

`new` দিয়ে call হয়েছে?

```
new Fn()
```

হলে:

```
this → new object
```

Step 3

`call/apply/bind` আছে?

```
fn.call(obj)  
fn.apply(obj)  
fn.bind(obj)
```

হলে explicit binding দেখো।

Step 4

Object method call?

```
obj.fn()
```

হলে:

```
this → obj
```

Step 5

কিছুই না?

Default binding দেখো।

Strict mode হলে:

```
this → undefined
```

64. Output Prediction — 1

```
const user = {  
  name: "Duke",  
  
  greet() {  
    console.log(this.name);  
  }  
};  
  
user.greet();
```

Answer:

```
Duke
```

কারণ:

```
user.greet()  
  ↓  
implicit binding  
  ↓  
this = user
```

65. Output Prediction — 2

```
"use strict";

function greet() {
  console.log(this);
}

greet();
```

Answer:

```
undefined
```

কারণ standalone regular function + strict mode ।

66. Output Prediction — 3

```
function greet() {
  console.log(this.name);
}

const user = {
  name: "Duke"
};

greet.call(user);
```

Answer:

```
Duke
```

কারণ explicit binding ।

67. Output Prediction — 4

```
function greet() {  
  console.log(this.name);  
}  
  
const user = {  
  name: "Duke"  
};  
  
const fn = greet.bind(user);  
  
fn();
```

Answer:

Duke

68. Output Prediction — 5

```
const user = {  
  name: "Duke",  
  
  greet: () => {  
    console.log(this.name);  
  }  
};  
  
user.greet();
```

এখানে `this` `user` হবে না।

কারণ arrow function-এর own `this` নেই।

69. Output Prediction — 6

```
const user = {  
  name: "Duke",
```

```

    greet() {
      setTimeout(() => {
        console.log(this.name);
      }, 0);
    }
  };

  user.greet();

```

Output:

Duke

কারণ:

```

greet()
↓
this = user
↓
arrow callback
↓
lexical this
↓
user

```

70. Output Prediction — 7

```

const user = {
  name: "Duke",

  greet() {
    setTimeout(function () {
      console.log(this.name);
    }, 0);
  }
};

user.greet();

```

এখানে callback regular function হওয়ায় outer `this` automatically inherit করবে না।

Environment/strictness অনুযায়ী behavior আলাদা হতে পারে, তাই exact browser/Node context না জেনে specific global value ধরে নেওয়া উচিত নয়।

মূল lesson:

Regular callback $\neq$ lexical `this`.

71. Output Prediction — 8

```
const user1 = {
  name: "Duke"
};

const user2 = {
  name: "Rahim"
};

function greet() {
  console.log(this.name);
}

const fn = greet.bind(user1);

fn.call(user2);
```

Output:

Duke

কারণ bound `this` সাধারণ `call()` দিয়ে replace করা যায় না।

72. Practical Rule — Method হলে Regular Function

যদি object method-এর নিজের object context দরকার হয়:

```
const user = {
  name: "Duke",

  greet() {
    console.log(this.name);
  }
};
```

```
}  
};
```

এই pattern সবচেয়ে পরিষ্কার।

73. Practical Rule — Callback হলে Arrow

যদি outer `this` preserve করতে হয়:

```
function User() {  
  this.name = "Duke";  
  
  setTimeout(() => {  
    console.log(this.name);  
  }, 1000);  
}
```

Arrow function useful।

74. Practical Rule — External Context Explicit করতে `call/apply/bind`

যখন manually context control করতে হবে:

```
fn.call(obj);
```

অথবা:

```
fn.apply(obj, args);
```

অথবা reusable bound function:

```
const boundFn = fn.bind(obj);
```

75. `call`, `apply`, `bind` — Quick Memory Trick

```
call
↓
Call now

apply
↓
Apply arguments as array-like

bind
↓
Bind now, call later
```

76. `this` Quick Reference

Situation	<code>this</code>
<code>obj.method()</code>	<code>obj</code>
<code>fn.call(obj)</code>	<code>obj</code>
<code>fn.apply(obj)</code>	<code>obj</code>
<code>fn.bind(obj)</code>	bound <code>obj</code>
<code>new Fn()</code>	newly created object
standalone regular function + strict mode	<code>undefined</code>
arrow function	lexical outer <code>this</code>
DOM regular event handler	current handler element
DOM arrow event handler	lexical <code>this</code>

77. Most Important Difference

Regular Function

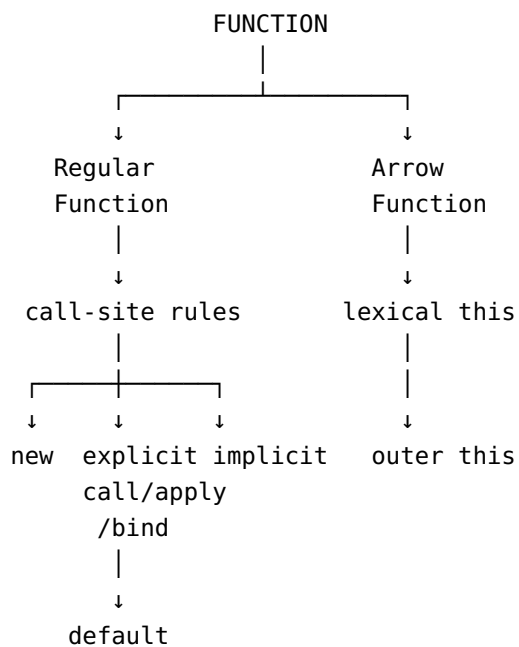
```
this
↓
call-site determined
```

Arrow Function

```
this
↓
lexical
↓
surrounding context থেকে আসে
```

এই difference পুরো section-এর heart ।

78. Final Mental Model



79. The 5 Rules — Final Revision

Rule 1 — Default

```
fn();
```

Standalone regular function |

Strict mode:

```
this → undefined
```

Rule 2 — Implicit

```
obj.fn();
```

```
this → obj
```

Rule 3 — Explicit

```
fn.call(obj);  
fn.apply(obj);  
fn.bind(obj);
```

```
this → explicitly provided object
```

Rule 4 — new

```
new Fn();
```

```
this → newly created instance
```

Rule 5 — Arrow

```
const fn = () => {};
```

নিজস্ব this নেই
↓
lexical this

80. Interview Questions

Fundamentals

1. JavaScript-এ `this` কী?
2. `this` কি lexical scope follow করে?
3. Regular function-এর `this` কীভাবে determine হয়?
4. Arrow function-এর `this` কীভাবে determine হয়?
5. `this` এবং normal variable-এর মধ্যে difference কী?

Binding

1. Default binding কী?
2. Implicit binding কী?
3. Explicit binding কী?
4. `new` binding কী?
5. Arrow function কেন আলাদা?

call/apply/bind

1. `call()` কী?
2. `apply()` কী?
3. `bind()` কী?
4. `call()` এবং `apply()` -এর difference কী?
5. `call()` এবং `bind()` -এর difference কী?
6. Function borrowing কী?
7. `bind()` কীভাবে partial application করতে পারে?

Callbacks

1. Callback-এর মধ্যে `this` কেন confusing?
2. Arrow function callback-এ কেন useful?
3. `setTimeout` callback-এ regular function বনাম arrow-এর difference কী?
4. `forEach()` -এর callback-এ `thisArg` কী?

5. Arrow function কি `thisArg` মানে?

DOM

1. DOM event handler-এ regular function-এর `this` কী?
2. `this` এবং `event.currentTarget` -এর difference কী?
3. `event.target` এবং `event.currentTarget` -এর difference কী?
4. Arrow event handler-এ `this` element হয় না কেন?

Advanced

1. Method extract করলে `this` কেন হারায়?
2. Bound function-এর `this` আবার `call()` দিয়ে change করা যায়?
3. Arrow function-এ `call()` / `apply()` / `bind()` কেন expectedভাবে `this` change করতে পারে না?
4. Class method callback হিসেবে pass করলে কী সমস্যা হতে পারে?

81. Practice Problems

Problem 1

Output কী হবে?

```
const user = {  
  name: "Duke",  
  
  greet() {  
    console.log(this.name);  
  }  
};  
  
user.greet();
```

Problem 2

Output কী হবে?

```
const user = {  
  name: "Duke",  
  
  greet() {  
    console.log(this.name);  
  }  
};
```

```
    }  
  };  
  
  const fn = user.greet;  
  
  fn();
```

Strict mode ধরে answer দাও।

Problem 3

```
function greet() {  
  console.log(this.name);  
}  
  
const user = {  
  name: "Duke"  
};  
  
greet.call(user);
```

`this` কোথায় bind হলো?

Problem 4

```
function greet(message) {  
  console.log(message, this.name);  
}  
  
const user = {  
  name: "Duke"  
};  
  
const fn = greet.bind(user, "Hello");  
  
fn();
```

Output কী?

Problem 5

```
const user = {
  name: "Duke",

  greet() {
    setTimeout(() => {
      console.log(this.name);
    }, 0);
  }
};

user.greet();
```

Arrow callback কেন `Duke` পায়?

Problem 6

```
const user = {
  name: "Duke",

  greet: () => {
    console.log(this.name);
  }
};

user.greet();
```

কেন `this` `user` নয়?

Problem 7

```
const user1 = {
  name: "Duke"
};

const user2 = {
  name: "Rahim"
};

function greet() {
```

```

    console.log(this.name);
  }

  const fn = greet.bind(user1);

  fn.call(user2);

```

Output কী এবং কেন?

82. Mastery Checklist

এই section শেষ হলে তোমার পারা উচিত:

- ☐ `this` কী explain করতে
- ☐ Call-site identify করতে
- ☐ Default binding বুঝতে
- ☐ Implicit binding বুঝতে
- ☐ Explicit binding বুঝতে
- ☐ `new` binding বুঝতে
- ☐ Arrow lexical `this` বুঝতে
- ☐ `call()` ব্যবহার করতে
- ☐ `apply()` ব্যবহার করতে
- ☐ `bind()` ব্যবহার করতে
- ☐ `call/apply/bind` compare করতে
- ☐ Function borrowing করতে
- ☐ Partial application বুঝতে
- ☐ Method extraction-এর সমস্যা বুঝতে
- ☐ Callback-এর `this` predict করতে
- ☐ `setTimeout` -এ `this` বুঝতে
- ☐ Array callback-এর `this` বুঝতে
- ☐ DOM event handler-এর `this` বুঝতে
- ☐ `event.target` vs `event.currentTarget` বুঝতে
- ☐ Regular vs arrow function-এর `this` difference explain করতে
- ☐ Code দেখে `this` predict করতে

83. Final Cheat Sheet

REGULAR FUNCTION

`obj.fn()`

```
→ this = obj

fn.call(obj)
→ this = obj

fn.apply(obj, args)
→ this = obj

fn.bind(obj)
→ new function with bound this

new Fn()
→ this = new instance

fn()
→ default binding
→ strict mode: undefined
```

ARROW FUNCTION

```
() => {}
→ নিজের this নেই
→ lexical this

call()
apply()
bind()
→ arrow-এর lexical this replace করতে পারে না
```

সবচেয়ে **important sentence:**

Regular function-এর `this` বুঝতে **function** কোথায় লেখা হয়েছে সেটা না দেখে **function** কীভাবে **call** হয়েছে সেটা দেখো। **Arrow function**-এর ক্ষেত্রে নিজের `this` নেই—**surrounding lexical this** ব্যবহার করে।

এটাই `this` mastery-এর core concept।